Operating System Capstone

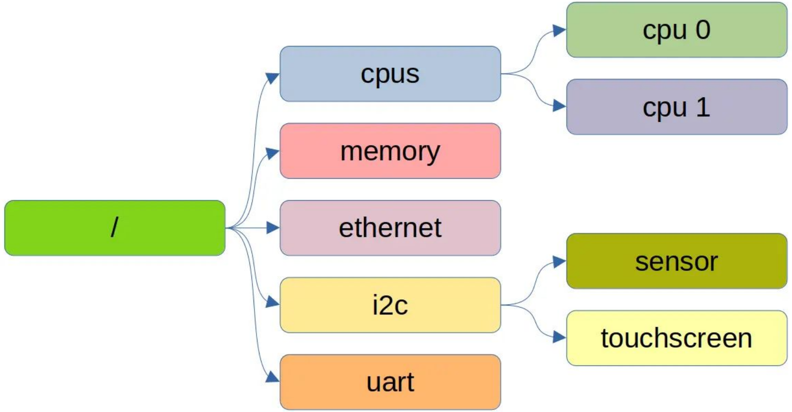
Lab2 Exercise

Overview

- **Ex 2-1. Devicetree Parsing**
- Ex 2-2. Initial Ramdisk Parsing

Introduction

- Devicetree
  - A data structure describing hardware, allowing the kernel to find devices without hardcoded addresses.
  - Get the specification here.



Introduction

- Format
  1. Devicetree source (.dts): human-readable form
  2. Flattened devicetree (.dtb): compiled format for parsing
- Flattened devicetree (.dtb) structure:

FDT Header	Definition
Memory Reservation Block	(Not used here)
Structure Block	Tokens with data
Strings Block	Property names

FDT Header
Memory Reservation Block
Structure Block
Strings Block

Devicetree Parsing

- FDT Header
  - All the header fields are 32-bit integers, stored in **big-endian** format.
  - Useful fields here:
    - magic (0xd00dfeed)
    - off_dt_struct
    - off_dt_strings

```
struct fdt_header {
    uint32_t magic;
    uint32_t totalsize;
    uint32_t off_dt_struct;
    uint32_t off_dt_strings;
    uint32_t off_mem_rsvmap;
    uint32_t version;
    uint32_t last_comp_version;
    uint32_t boot_cpuid_phys;
    uint32_t size_dt_strings;
    uint32_t size_dt_struct;
};
```

FDT Header
Memory Reservation Block
Structure Block
Strings Block

Devicetree Parsing

- Structure Block
  - Padding: align up to 4 bytes

Tag	ID	Description
FDT_BEGIN_NODE	0x1	Node_name with '\0' **|** Padding
FDT_END_NODE	0x2	End of the node
FDT_PROP	0x3	Prop_len **|** Prop_name offset in Strings Block **|** Padding
FDT_NOP	0x4	Ignored
FDT_END	0x9	End of the devicetree

Todo

- **fdt_path_offset**
  - Return the offset of a node by path (the first match one).
- **fdt_getprop**
  - Return the pointer to property data.

```c
int fdt_path_offset(const void* fdt, const char* path) {
    // TODO: Implement this function
}

const void* fdt_getprop(const void* fdt,
                        int nodeoffset,
                        const char* name,
                        int* lenp) {
    // TODO: Implement this function
}
```

Testcases

- Already written in main:

Node Offsets to Find	Properties to Get
/cpus/cpu@0/interrupt-controller	compatible
/memory	reg
/chosen	linux,initrd-start

- Expected result:

```
compatible: riscv,cpu-intc
memory: base=0x80000000 size=0x200000000
initrd-start: 0xa0200000
```

Overview

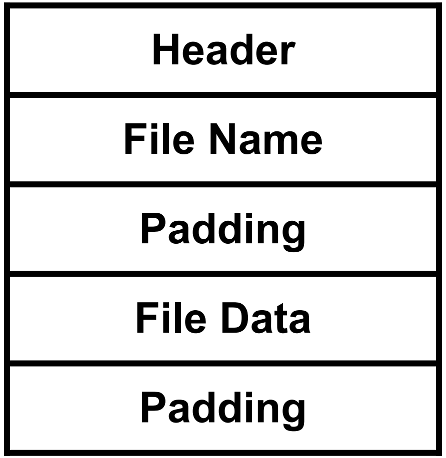
- Ex 2-1. Devicetree Parsing
- **Ex 2-2. Initial Ramdisk Parsing**

Introduction

- Initial Ramdisk (initrd)
    - A temporary root file system loaded into memory during the boot process.
    - Provides essential files and drivers needed to mount the actual root file system.
    - We use a **cpio archive** to serve as the initrd in our lab. See the manual here.

New ASCII Format Cpio Archive

- Archive Structure
  - Padding: align up to 4 bytes



New ASCII Format Cpio Archive

- Header
  - Useful fields here:
    - magic ("070701")
    - filesize
    - namesize

```
struct cpio_newc_header {
    char magic[6];
    char ino[8];
    char mode[8];
    char uid[8];
    char gid[8];
    char nlink[8];
    char mtime[8];
    char filesize[8];
    char devmajor[8];
    char devminor[8];
    char rdevmajor[8];
    char rdevminor[8];
    char namesize[8];
    char check[8];
};
```

New ASCII Format Cpio Archive

- Archive Stream
  - The end of the archive is indicated by a special record with the pathname **"TRAILER!!!"**

Header	**Header**	**Header**
"FileA"	**"FileB"**	**"TRAILER!!!"**
Padding	**Padding**	**(End)**
A's content	**B's content**	
Padding	**Padding**	

Todo

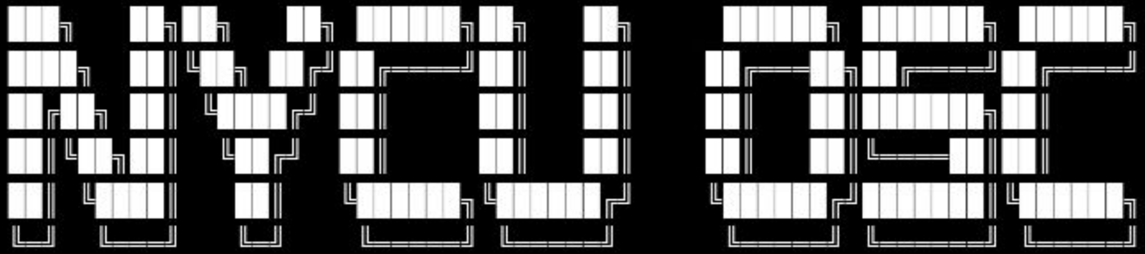
- **initrd_list**
  - List files in the initrd.
- **initrd_cat**
  - Display file contents in the initrd.

```c
void initrd_list(const void* rd) {
    // TODO: Implement this function
}

void initrd_cat(const void* rd, const char* filename) {
    // TODO: Implement this function
}
```

Expected Result

```
     0 .
   971 osc.txt
    14 author.txt
   337 penguin.txt
395441 osctest.bin
```




```
initrd_cat: test.txt: No such file
```